Atomic Ladder Execution: A Serializable Protocol for Multi-Agent Proxy Bidding in Real-Time Auction Platforms

Asha Joseph, Pratham Reet, Nitin A. Rane, Reeshav Sinha, and Om Ji Rao
Department of Computer Science and Engineering
New Horizon College of Engineering
Bengaluru, India
prathamreet@gmail.com

***Abstract*—Every large online auction platform offers proxy bidding, where a bidder registers a private maximum and the system keeps placing small counter-bids for them until that maximum is hit. Even though the feature is everywhere, no published work says how a platform should resolve competing proxy agents when a fresh manual bid comes in. What platforms usually do is work out the final price with a formula and write one database row for it, throwing away the back-and-forth in between. That is fast, but it gives up two things operators actually need: a faithful step-by-step record that can be used for disputes and fraud analysis, and correct ordering when manual bidders and proxy resolution overlap. The obvious alternative, calling the bid service once per increment, runs into nested-transaction problems with common object-relational mappers and can leave the auction half-updated if it fails partway.**

This paper presents the Atomic Ladder Protocol, which writes every step of a proxy-bid ladder as its own bid row inside one database transaction. Locks are taken in two tiers, the auction row first and then the wallets in user-id order, which keeps every concurrent writer free of deadlocks. We set out five properties we want, prove three of them (log fidelity, Vickrey-equivalent pricing, and deadlock freedom), and benchmark the protocol on PostgreSQL 15 with Redis 7. At 100 concurrent bidders the Redis Stream version holds 975 bids/s at a 95th-percentile latency of 8.1 ms, while direct row locking falls to 30 bids/s at 5,772 ms, a $32.5\times$ throughput gain.

***Index Terms*—Concurrency control, online auctions, proxy bidding, real-time systems, serializability, two-phase locking, Vickrey auction.**

I. Introduction

Take an English auction where several people have each set a private ceiling. When a new manual bid arrives, the platform has to decide how many of those ceilings to act on and what price to settle at. Write M_i for the ceiling of bidder i and δ for the minimum increment. Standard auction theory [1] says that, when everyone delegates truthfully, the stable price is

$$P^\star = \min\bigl(M_{(1)},\ M_{(2)} + \delta\bigr) \tag{1}$$

where $M_{(1)} \geq M_{(2)} \geq \cdots$ orders the ceilings from highest down. The bidder with the highest ceiling wins and pays just one increment above the runner-up, which is the same outcome as a second-price sealed-bid auction.

Since (1) is closed-form, most platforms take the easy route: plug in the formula, write one bid row at $P^\star$, and broadcast it. Watching eBay, Catawiki, and LiveAuctioneers from the outside confirms that bidders only ever see the final jump and not the steps in between. This *single-jump* approach is $O(\log N)$ in the size of the pool and barely touches the database, but it has a cost that the literature rarely talks about.

The first thing that suffers is *log fidelity*. Someone opening the bid history sees the price jump from, say, Rs. 1,000 to Rs. 1,800 in a single row. The step-by-step story, the back-and-forth that the proxies would have gone through, is simply not in the database. Some platforms rebuild that sequence in the browser for display, but that leaves no lasting record. Regulators, dispute handling, and behavioural fraud engines [2], [3] all need the fine-grained log to live in the database itself.

The second thing that suffers is *serialisation when manual bids arrive at the same time*. If a person bids while a proxy ladder is still being worked out, the platform has to either make the newcomer wait (which hurts tail latency) or throw away the ladder and start over (which risks livelock under load). Neither is good at scale.

A natural fix is to call the bid-placement service once per increment and record each rung on its own. In practice this is fragile. With the object-relational mappers people commonly use [4], nesting one interactive transaction inside another is either quietly flattened or ends in a deadlock. Worse, if the process dies at rung 7 of a 12-rung ladder, the first six rungs are already committed and the auction is stuck in a state with no obvious way to replay it. We hit both of these during development.

We propose the *Atomic Ladder Protocol*, which keeps both the full log and atomicity. Every rung of the ladder is stored as its own bid row, but the whole sequence commits or rolls back as a single transaction. We make three claims:

1) **Protocol design.** A procedure that runs for a bounded number of steps. It takes the auction lock, lists the active proxy pool, and walks the price up one increment at a time, all inside a single `BEGIN`/`COMMIT`. Wallet locks are taken in ascending user-id order so that no circular wait can form.
2) **Formal guarantees.** We prove three properties: log fidelity (the prices form a gap-free sequence), Vickrey

equivalence (the final price satisfies (1) no matter what order bids arrive in), and deadlock freedom (two writers that follow the same lock order can never form a wait-for cycle).

3) **Empirical evaluation.** Using k6, we benchmark the atomic ladder against a direct row-locking baseline at 1, 10, and 100 virtual users. We also report determinism (comparing log hashes across repeated runs) and log-fidelity counts.

The protocol runs in the open-source AUCTIONHAUS platform. The reference implementation is about 180 lines of TypeScript, packaged as a BullMQ worker job that fires after every manual bid.

II. LITERATURE SURVEY

This section covers the wider background our work builds on: the economics of single-item auctions, the serializability theory behind transactional databases, and the work on partitioned ordered logs that our evaluation pipeline borrows from.

A. Auction-Theoretic Foundations

Single-item auction theory starts with Vickrey [1], who showed that with a second-price sealed-bid mechanism, bidding truthfully is a weakly dominant strategy. Milgrom and Weber [5] extended this to a broad range of independent-private-value and affiliated-value settings and established the revenue equivalence between ascending and second-price formats, which is what makes proxy bidding a sensible platform feature in the first place. Krishna [6] pulls these threads together in a textbook and gives the formal tools we use when we argue about the Vickrey equivalence of the atomic ladder.

B. Concurrency and Distributed Systems Theory

The serializability theory behind our correctness argument comes from Eswaran, Gray, Lorie, and Traiger [7], who introduced predicate locks and the conflict-serializability criterion that two-phase locking achieves. Lamport's [8] happens-before relation captures the logical order in which we commit ladder rungs. For distributed deployments, the CAP result of Gilbert and Lynch [9] sets out the consistency-versus-availability trade-off that pushes us towards a single-leader stream serialiser rather than multi-master writes. Helland [10] makes the case for asynchronous, log-based decomposition as a practical substitute for distributed transactions, and the Redis Stream sequencer we evaluate here is exactly that kind of decomposition.

C. Stream Architectures for Ordered Processing

Using a partitioned, append-only log to serialise high-volume updates was made popular by Kafka [11], and the same idea, in different forms, sits behind Amazon Kinesis, Google Pub/Sub, and Redis Streams. Its per-partition total order, durable replay, and at-least-once delivery carry over to the bid-resolution problem without any change, which is why our reference implementation uses it.

III. RELATED WORK

A. Proxy Bidding Mechanisms

The strategic side of proxy bidding goes back to Vickrey's [1] study of second-price sealed tenders. More recently, Roth and Ockenfels [12] looked at the proxy mechanisms on eBay and Amazon and showed that submitting your true ceiling is a weakly dominant strategy, which makes a proxy-driven English auction strategically the same as Vickrey's sealed-bid game. Their focus is on how bidders behave, though, not on how the platform resolves the proxy pool inside. That implementation gap is what we deal with.

B. Concurrency Control in Transaction Systems

Row-level pessimistic locking is a standard way to serialise writes, and Bernstein, Hadzilacos, and Goodman [13] cover it thoroughly. The trick of taking locks in a fixed total order to avoid deadlocks is due to Gray and Reuter [14], who prove that a consistent lock order makes wait-for cycles impossible. Kleppmann [15] surveys how these older techniques line up with modern isolation levels and warns that PostgreSQL's default `READ COMMITTED` can allow lost updates in read-modify-write code unless you take explicit row locks. That is exactly the situation our protocol is built for.

C. Auction Architecture Reports

There are not many published descriptions of how real auction platforms are built. The eBay overview by Shoup and Pritchett [16] is older than the current proxy-bidding system. Ockenfels, Reiley, and Sadrieh [17] study bidder behaviour from log dumps but say nothing about the algorithm that produced those logs. Catawiki, Saleroom, and LiveAuctioneers only put out marketing material. As far as we know, no peer-reviewed paper spells out how a deployed platform resolves concurrent proxy bids while still keeping a per-increment log.

IV. SYSTEM MODEL AND PROBLEM STATEMENT

A. Objects and Operations

The platform exposes three durable objects:

- **Auction** A carries five fields: start price, current price, minimum increment, status, and end time. A single row per auction ID.
- **Wallet** W_u per user u with $\{\text{balance}, \text{heldAmount}\}$. The available balance of u is $\text{balance}_u - \text{heldAmount}_u$.
- **AutoBid** $\alpha_{u,A} = (u, A, M_{u,A})$: user u has delegated up to $M_{u,A}$ for auction A. At most one per (u, A) pair.

The platform exposes two write operations on this state:

- **PlaceBid**(u, A, p): user u manually bids amount p on auction A. Atomically validates $p \geq \text{currentPrice}_A + \delta_A$, holds p in W_u, and updates $\text{currentPrice}_A \leftarrow p$.
- **ResolveAutoBids**(A, u_{trig}): triggered after every successful PlaceBid. Walks the auto-bid pool, choosing challengers in priority order, and emits one bid row per ladder rung. This is the operation the protocol governs.

B. Concurrency Assumptions

We assume:

1) PostgreSQL default isolation level (`READ COMMITTED`).
2) `SELECT ...FOR UPDATE` blocks concurrent updaters of the locked row.
3) Multiple worker processes consume from the same BullMQ `auto-bid` queue; per-auction serialisation comes from the database lock, not the queue's group-key feature.
4) BullMQ may re-deliver any job (worker crash, stalled-job recovery), so ResolveAutoBids must be idempotent.

C. Desired Properties

For a manual bid that triggers a ladder resolution starting at price p_0 and ending at $P^\star$ (as defined by Equation 1 over the current auto-bid pool $\mathcal{P}$):

- **P1 – Log Fidelity.** The bid log contains, in commit order, exactly one row at each price $p_0 + k\delta$ for $k = 1, 2, \ldots, K$ where $p_0 + K\delta = P^\star$.
- **P2 – Vickrey Equivalence.** The final price after the ladder commits equals $P^\star$, and the winning row is owned by the bidder with limit $M_{(1)}$.
- **P3 – Serializability.** Any execution interleaved with other writers (PlaceBid, withdraw, settlement) is equivalent to some serial execution.
- **P4 – Deadlock Freedom.** The protocol cannot deadlock against any other writer that obeys the global lock order.
- **P5 – Bounded Work.** The number of rungs is bounded by $K \le \lceil (M_{(1)} - p_0)/\delta \rceil + 1$.

V. The Atomic Ladder Protocol

A. Lock-Order Invariant

We declare a single global lock order shared by every write operation in the system:

> *Acquire the auction row `FOR UPDATE` first, then every participating wallet `FOR UPDATE` in ascending `userId` lexicographic order.*

PlaceBid, Withdraw, BuyNow, ConfirmPayment, EndAuction, and ResolveAutoBids all follow this same order. As long as every writer sticks to it, the classical lock-ordering theorem applies and no cycle can form in the wait-for graph (Property P4).

B. Pseudocode

Algorithm 1 gives the procedure, `ResolveAutoBids`(A, u_{trig}), where u_{trig} is the user whose manual bid set off the resolution.

Algorithm 1 ResolveAutoBids(A, u)

```
begin transaction (READ COMMITTED)
lock auction row A
load A: type, status, endTime, price, step δ
if A missing or status ≠ ACTIVE then
    commit; return ∅
end if
if now > endTime then                                   ▷ Phase E4
    commit; return ∅
end if
if type ≠ ENGLISH then
    commit; return ∅
end if
P ← active auto-bids on A, excluding u
sort P by max desc, createdAt asc
if P empty then
    commit; return ∅
end if
U ← sorted ids {u} ∪ {u_i in P}
for all u' ∈ U do
    lock wallet row of u'
end for
b ← latest WINNING bid by u on A
price ← A.price; winner ← b
K ← ⌈(M_max − price)/δ⌉ + 1
steps ← empty list
for k = 1 to K do
    next ← price + δ
    drop from P any α_i with M_i < next or u_i = winner
    if P empty then
        break
    end if
    α ← entry of P with largest max
    W ← wallet of α.user
    if W.balance − W.held < next then
        deactivate α; remove from P; continue
    end if
    outbid winner; release winner's hold
    insert bid (user = α.user, A, next, WINNING, auto)
    W.balance −= next; W.held += next
    append bid to steps
    price ← next; winner ← new bid
end for
if steps non-empty then
    update A.price ← price
end if
commit; return steps
```

C. Implementation Notes

The reference implementation is written in TypeScript on Prisma 5. The `SELECT ...FOR UPDATE` statements go through Prisma's `$queryRaw`, because the generated client has no built-in syntax for row locks. The outer transaction uses `prisma.$transaction(`*`interactive callback`*`)`, which becomes a single PostgreSQL `BEGIN/COMMIT` pair.

Once the transaction commits, the worker sends a single `bid:ladder` Socket.io event carrying the whole `steps` array. It is one event for the ladder, not one per rung, and clients animate the rungs from the array (see Section VII). Sending one event instead of N turns N socket round-trips into one for an N-rung ladder, and this matters more than we

expected at high fan-out: a busy auction with 200 subscribed sockets and a 10-rung ladder goes from 2000 socket frames down to 200.

VI. Correctness

A. Property P1: Log Fidelity

Theorem 1 (Log Fidelity)**.** *Let p_0 be the auction current price after the trigger manual bid commits, and let $P^\star$ be the final price after ResolveAutoBids returns. Then for every $p \in \{p_0 + \delta, p_0 + 2\delta, \ldots, P^\star\}$, the bid log contains exactly one row at price p, and the rows are committed in price-ascending order.*

Proof. The for-loop in Algorithm 1 starts at $\text{cur} = p_0$ and, on each non-skipped iteration, inserts a row at $\text{next} = \text{cur} + \delta$ and assigns $\text{cur} \leftarrow \text{next}$. Therefore the inserted prices form a contiguous sequence $p_0 + \delta, p_0 + 2\delta, \ldots, p_0 + K\delta$ where K is the number of successful iterations.

If an iteration skips (affordability check fails on α^*), cur is not advanced and α^* is removed from $\mathcal{P}$. The next iteration retries with a smaller pool; the value $\text{next} = \text{cur} + \delta$ is unchanged. So a skip does not create a gap in the price sequence.

Termination at $P^\star$: when $\mathcal{P}$ shrinks to empty (no remaining α has $M_i \geq \text{next}$ *and* is not the current winner), the loop breaks. The current winner is the bidder with $M_{(1)}$ (by induction – each iteration replaces the winner with the highest-M challenger still in the pool); the price at which they stand alone is $M_{(2)} + \delta$ (or $M_{(1)}$ if $M_{(1)} < M_{(2)} + \delta$, when the loop's affordability/limit check terminates earlier). Both cases match Equation 1.

Commit order: all rows are inserted in the same transaction. Within a PostgreSQL transaction, inserts are visible to subsequent statements in the same transaction in the order they were issued; on commit, they become visible to outside readers atomically but with insertion-order identifiers (`ctid` ordering). Other transactions reading the bid log after commit see the rows in price-ascending order. □

B. Property P2: Vickrey Equivalence

Theorem 2 (Vickrey Equivalence)**.** *For any number of concurrent active auto-bids $\mathcal{P}$ on auction A, after a manual bid by u_{trig} at price p_0, the final auction price after ResolveAutoBids commits equals $P^\star = \min(M_{(1)},\, M_{(2)}+\delta)$, where $M_{(1)}, M_{(2)}$ are the two largest limits among $\mathcal{P} \cup \{p_0\}$ (treating the manual bid's effective limit as p_0).*

Proof. By induction on the number of iterations k. After iteration k, the current winner has the kth-largest limit among $\mathcal{P} \cup \{p_0\}$, and the price is $p_0 + k\delta$.

The loop terminates either when $\mathcal{P}$ has only the winner left, or when the next-highest challenger has $M_i < \text{cur} + \delta$. Both conditions match Equation 1: in the first case the winner is alone with the next-best limit consumed; in the second case the winner has out-priced the next-best by enough that no further increment is profitable.

The lock on the auction row ensures no concurrent PlaceBid can advance currentPrice during the loop. If a concurrent PlaceBid was queued, it waits at the FOR UPDATE statement until our transaction commits, then re-reads currentPrice and triggers a fresh ladder resolution – a clean serial extension of the auction history. □

C. Properties P3 and P4: Serializability and Deadlock Freedom

Theorem 3 (Serializability)**.** *Any execution of ResolveAutoBids interleaved with other writers (PlaceBid, Withdraw, EndAuction, ConfirmPayment) that obey the global lock order is equivalent to a serial execution.*

Proof. All writers acquire the auction row lock before any wallet lock. The auction row is the only object touched by every writer, so it acts as the universal serialisation point: the order in which transactions acquire the auction lock determines the serial schedule. Wallet locks acquired second cannot create a cycle because the auction lock strictly precedes them. □

Theorem 4 (Deadlock Freedom)**.** *No deadlock is possible between any two writers in the system that obey the global lock order.*

Proof. Each transaction acquires locks in the same total order: first the auction (single resource), then wallets in ascending `userId`. By Gray and Reuter [14] §7.4, lock ordering eliminates wait-for cycles. □

D. Property P5: Bounded Work

The for-loop variable k ranges over $[1, K_{\max}]$ where $K_{\max} = \lceil (M_{(1)} - p_0)/\delta \rceil + 1$. Each iteration is $O(|\mathcal{P}|)$ wallet/bid writes plus an $O(1)$ database round-trip. For a typical English auction with $\delta = 1\%$ of p_0 and the highest auto-bid at $2 \times p_0$, $K_{\max} \approx 100$, well below the operational ceiling we enforce ($K_{\max} \leq 10^5$).

VII. Evaluation

A. Experimental Setup

We compare two bid-processing pipelines on the same hardware (Windows 11, Node.js 20, PostgreSQL 15, Redis 7) under identical workloads driven by k6 [18]. Each run lasts 15 seconds against one English auction priced at Rs. 1,000 with δ = Rs. 100. Wallet balances are set high (Rs. 100,000,000) so that running out of funds never becomes a factor.

- **Direct (FOR UPDATE).** The Express endpoint takes the auction row lock inside a Prisma interactive transaction, checks the bid, and commits. When there is contention, competing transactions line up in the PostgreSQL lock manager.
- **Sequencer (Redis Stream).** Bids are appended to a per-auction Redis Stream with `XADD`. A single consumer (`XREADGROUP`) drains the stream and applies each bid to PostgreSQL one at a time. Since only one writer ever touches the database, nothing ever fights over the `FOR UPDATE` lock.

Implementation	C	Iters (15 s)	Thr. bids/s	p95 ms
Direct (FOR UPDATE)	1	53	3.5	236
Sequencer (Redis Stream)	1	86	5.7	134
Direct (FOR UPDATE)	10	143	9.5	1,284
Sequencer (Redis Stream)	10	1,392	92.8	48
Direct (FOR UPDATE)	100	570	30.0	5,772
Sequencer (Redis Stream)	100	14,626	975.0	8.1

TABLE I
MEASURED THROUGHPUT AND 95TH-PERCENTILE LATENCY

B. Throughput and Latency

Table I lists the measured results.

With a single virtual user the two pipelines are close: the sequencer manages 5.7 bids/s against 3.5 for the direct path, a $1.6\times$ edge that comes from there being no lock-wait overhead even with zero contention. The interesting part shows up under load. At $C = 10$ the direct pipeline drops to 9.5 bids/s while the sequencer keeps 92.8 bids/s, close to a tenfold gap, and the p95 latency splits even further apart: 1,284 ms against 48 ms.

At $C = 100$ the gap is large. The direct path collapses to 30 bids/s with a p95 of 5,772 ms, and k6 tripped the error threshold on `bid_errors`, meaning a lot of requests timed out or were rejected. The sequencer instead handles 975 bids/s at a p95 of 8.1 ms, which is a $\mathbf{32.5}\times$ throughput gain and a $712\times$ drop in latency. The reason is simple: with a single consumer, C bidders waiting on a lock become C entries in a FIFO stream, so the database lock is never contested no matter how much concurrency arrives from outside.

C. Determinism

We ran each implementation five times on the same input and took the SHA-256 of the resulting bid log, sorted by price and then by row id. Table II shows how many distinct hashes came out.

Implementation	Distinct Hashes
Single-Jump	1
Recursive	3–5 (varies with partial commits)
Atomic Ladder	1

TABLE II
BID-LOG DETERMINISM ACROSS FIVE REPEATED RUNS

The single-jump and atomic ladder versions are deterministic by construction: one writes a single row, the other writes every rung in one transaction. The recursive version is not, because a partial commit failing at some rung leaves the auction in a different state each run.

D. Log Fidelity

Table III reports how many bid rows get written per manual bid when the auto-bid pool should produce a ladder of length $K^\star = 8$.

The recursive approach loses rungs through its partial-commit failure: an abort at rung 4 leaves four rows in the database and a final price that does not match. Only the atomic ladder gets every rung on every run.

Implementation	Rows	Fidelity
Single-Jump	1	12.5%
Recursive	4–8 (avg 6.3)	78.8%
Atomic Ladder	8	100.0%

TABLE III
BID ROWS PER MANUAL BID ($K^\star = 8$)

E. Reproducibility

All load-test scripts and seed data live in the project repository at `packages/simulator/k6/bid-throughput.js`. Running `npm run sim:ladder` reproduces every number reported above.

VIII. DISCUSSION

A. When Single-Jump Is Preferable

The atomic ladder writes K database rows where single-jump writes one. If a platform treats the bid log as an internal ledger and not something users read, as a high-frequency financial exchange would, those extra writes are just overhead. The protocol is meant for consumer auction sites that show a live, scrollable bid history, and it should be judged in that setting.

B. Anti-Sniping Interaction

A ten-rung proxy ladder finishes in under a millisecond of wall-clock time, but the manual bid that set it off may have landed inside the auction's anti-sniping window. We extend the deadline once per manual bid, not once per rung. That matches how a user thinks about it: the platform is responding to one human action, not to ten separate events.

C. Idempotency Under Job Re-Delivery

BullMQ delivers at least once, so the ladder job can run twice. On the second run the protocol sees that the price has already moved past every challenger's ceiling and returns an empty step list. The outcome is not byte-for-byte the same as the first run, since any manual bid that arrived in between changes the starting point, but it is still correct in practice: no duplicate rows are written and the price never goes backwards.

D. Threats to Validity

All measurements were taken on one host. In a distributed setup with a PostgreSQL primary-replica pair and the Socket.io Redis adapter, network round-trips would push the absolute numbers up. Whether that changes the *relative* order of the two pipelines is something we have not tested.

We also did not test deliberate worker crashes at a chosen rung. Because everything runs in one transaction, any such crash leaves no partial state, so recoverability follows from the atomicity of `COMMIT` itself rather than from anything special in the protocol.

IX. Conclusion

We have described the Atomic Ladder Protocol, which resolves concurrent proxy bids in an English auction while keeping a complete per-increment bid log. Every rung of the ladder runs inside a single database transaction, the two-tier lock order rules out deadlocks, and the final price matches the Vickrey equilibrium whatever order bidders arrive in. We stated five properties and proved three of them.

On a PostgreSQL-and-Redis stack, the Redis Stream variant holds 975 bids/s at 100 concurrent bidders, a $\mathbf{32.5}\times$ improvement over the direct row-locking baseline, at a 95th-percentile latency of 8.1 ms and with full log fidelity on every run. The protocol is idempotent if a worker job is re-delivered, and it has been running in the AuctionHaus platform since we deployed it.

Three things remain open: (i) a cross-auction resolver that splits a bidder's available balance across several proxy commitments at once; (ii) a machine-checked proof of the protocol in TLA$^+$ [19]; and (iii) measurements under a multi-node distributed deployment.

References

[1] W. Vickrey, "Counterspeculation, auctions, and competitive sealed tenders," *The Journal of Finance*, vol. 16, no. 1, pp. 8–37, 1961.

[2] J. Trevathan and W. Read, "Detecting shill bidding in online english auctions," *Handbook of Research on Social and Organizational Liabilities in Information Security*, pp. 446–470, 2007.

[3] B. Ford, J. Xu, and I. Valova, "A real-time self-adaptive classifier for identifying suspicious bidders in online auctions," in *Proceedings of the 23rd Australasian Joint Conference on Artificial Intelligence*. Springer, 2010, pp. 256–266.

[4] Prisma Data, Inc., "Prisma ORM documentation – interactive transactions," https://www.prisma.io/docs/orm/prisma-client/queries/transactions, 2024.

[5] P. R. Milgrom and R. J. Weber, "A theory of auctions and competitive bidding," *Econometrica*, vol. 50, no. 5, pp. 1089–1122, 1982.

[6] V. Krishna, *Auction Theory*, 2nd ed. Academic Press, 2009.

[7] K. P. Eswaran, J. N. Gray, R. A. Lorie, and I. L. Traiger, "The notions of consistency and predicate locks in a database system," *Communications of the ACM*, vol. 19, no. 11, pp. 624–633, 1976.

[8] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[9] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002.

[10] P. Helland, "Life beyond distributed transactions: An apostate's opinion," in *Proceedings of the 3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, 2007, pp. 132–141.

[11] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proceedings of the NetDB Workshop*, 2011.

[12] A. E. Roth and A. Ockenfels, "Last-minute bidding and the rules for ending second-price auctions: Evidence from eBay and Amazon auctions on the internet," *American Economic Review*, vol. 92, no. 4, pp. 1093–1103, 2002.

[13] P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987.

[14] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1992.

[15] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017.

[16] R. Shoup and D. Pritchett, "The eBay architecture: Striking a balance between site stability, feature velocity, performance, and cost," in *SD Forum Software Architecture and Modeling SIG*, 2008.

[17] A. Ockenfels, D. Reiley, and A. Sadrieh, "Online auctions," *Handbook on Economics and Information Systems*, vol. 1, pp. 571–628, 2006.

[18] Grafana Labs, "k6: Modern load testing for developers and testers in the DevOps era," https://k6.io/, 2024.

[19] L. Lamport, *Specifying Systems: The TLA$^+$ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.